

ZoomSense: A Scalable Infrastructure for Augmenting Zoom

Tom Bartindale, Peter Chen, Harrison Marshall, Stanislav Pozdniakov, Dan Richardson

Monash University
Melbourne, Australia

{tom.bartindale,peter.chen,hmar0007,stanislav.pozdniakov,dan.richardson}@monash.edu

ABSTRACT

We have seen a dramatic increase in the adoption of teleconferencing systems such as Zoom for remote teaching and working. Although designed primarily for traditional video conferencing scenarios, these platforms are actually being deployed in many diverse contexts. As such, Zoom offers little to aid hosts' understanding of attendee participation and often hinders participant agency. We introduce *ZoomSense*: an open-source, scalable infrastructure built upon 'virtual meeting participants', which exposes real-time meta-data, meeting content and host controls through an easy to use abstraction - so that developers can rapidly and sustainably augment Zoom.

CCS CONCEPTS

- **Human-centered computing** → **Interactive systems and tools**;
- **Information systems** → *Multimedia streaming*.

KEYWORDS

multimedia systems, experience, Zoom, infrastructure

ACM Reference Format:

Tom Bartindale, Peter Chen, Harrison Marshall, Stanislav Pozdniakov, Dan Richardson. 2021. ZoomSense: A Scalable Infrastructure for Augmenting Zoom. In *Proceedings of the 29th ACM International Conference on Multimedia (MM '21), October 20–24, 2021, Virtual Event, China*. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/3474085.3478332>

1 INTRODUCTION

The widespread expansion of remote working due to COVID-19 restrictions during 2020 has brought the design and use of these remote communication platforms into the spotlight, as whole industries and much of the education sector transitioned to exclusively remote working, with little-to-no warning. As a result, the use of web conferencing services has grown 87% over the course of 2020 as workers, educators and friends require new mediums through which to continue existing practices [1]. The Zoom videoconferencing platform has seen an 418% growth [1] in 2020, becoming a widely favoured tool for remote video collaboration [2, 4]. Unfortunately, this usage has exposed a number of limitations in the Zoom platform, particularly with regards to breakout (BO) rooms: the only users who are able to gain any insight into the activities occurring within breakout rooms are those who already reside within

them, leaving hosts who wish to monitor more than one group at a time in the dark about participants' engagement. Furthermore, participants lack much agency in moving between breakout rooms, or communicating with the host or other participants outside of their current room, as might be afforded in a physical environment. The technical limitations and available functions of the official Zoom API and SDK¹ necessitate that interaction with currently running meetings be carried out via a customised Zoom client application, offering little scope for developing new research prototypes built upon Zoom. In response, we developed *ZoomSense*: an open-source, scalable infrastructure designed to support **rapid development of augmentations to Zoom**, which already underpins active research projects in areas such as Learning Analytics [3]. Through a novel use of *Virtual Participants*, ZoomSense overcomes many of the inherent restrictions placed upon users and developers of the Zoom platform: it allows for the provision of real-time information to external services about participation in meetings, and the direct manipulation of Zoom meeting controls and chat from external sources. Our approach does not require customised software to be installed on participant or host devices, allowing it to be deployed on-demand at scale from a centralised server infrastructure. This allows an organisation to maintain data security, centralises resource management, and provides the flexibility to rapidly develop and deploy augmentations to the Zoom infrastructure.

2 ZOOMSENSE

ZoomSense is a cloud-based infrastructure which integrates with existing scheduled Zoom meetings (Figure 1). It uses virtual Zoom participants (dubbed ZoomSensors) to provide real-time data about attendee participation both inside and outside breakout rooms, while recording the video and audio within each breakout room for later analysis. Multiple can ZoomSensors automatically attend scheduled Zoom meetings and, once placed within a breakout room by the host, report participation information in real-time to the ZoomSense infrastructure which can be consumed and manipulated by external augmentations. ZoomSense is designed to: (1) Provide a sustainable and scalable solution for deploying ZoomSensors (2) Capture real-time information about participant engagement (3) Support extended forms of dialog between participants (4) Capture, store and process activity data with thought to consent and privacy. It consists of four key components:

2.1 ZoomSensor

<https://gitlab.com/action-lab-aus/zoomsense/zoomsense-windows>

The 'ZoomSensor' component is used to create a Virtual Participant within the Zoom call. No additional software needs to be installed by meeting attendees for ZoomSense to function: each is

¹<https://marketplace.zoom.us/docs/sdk/native-sdks/introduction>



This work is licensed under a Creative Commons Attribution International 4.0 License.

MM '21, October 20–24, 2021, Virtual Event, China.

© 2021 Copyright held by the owner/author(s).

ACM ISBN 978-1-4503-8651-7/21/10.

<https://doi.org/10.1145/3474085.3478332>

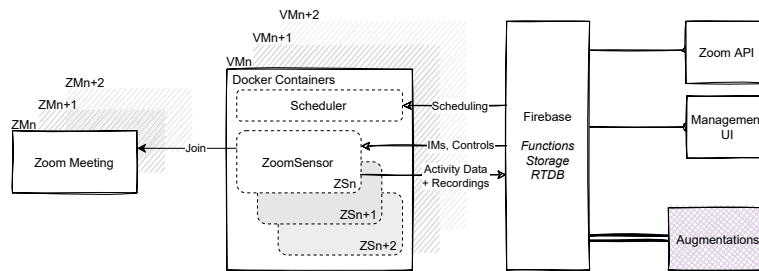


Figure 1: The high-level ZoomSense architecture: in order to augment multiple Zoom meetings, one or more VMs can each create one or more ZoomSensors, which each attend a single breakout room. Multiple ZoomSensors in multiple meetings are supported simultaneously. Rapid augmentation is supported primarily through interaction directly with the Firebase RTDB.

implemented as an instance of the ZoomSensor Windows Presentation Foundation application, developed using the official Zoom Windows SDK running inside a Windows Server Core docker container running in a cloud VM. Each ZoomSensor joins the Zoom call as a single attendee, and is placed into breakout rooms by the host like a normal meeting participant (Figure 2). Once joined, the ZoomSensor hooks into the Zoom SDK to monitor the room for Zoom events (e.g. ‘active speaker’ changes, generated by Zoom’s internal heuristic related to activity from participant audio), communicating this with the Firebase Real-time Database (RTDB) and observing it for any actions that need to be carried out (e.g. sending messages to individuals or groups). Each has bi-directional access to the Zoom chat room associated with the meeting or breakout room, allowing it to send and receive group and private messages to and from other attendees. These messages are passed to the RTDB, and as a result, it is possible for attendees to engage in text dialogue with the ZoomSense infrastructure. In the default implementation, this is used for transmitting a link to the default real-time visualisation to hosts and allowing attendees to instruct the ZoomSensor to disengage from their breakout room (and any data collection or recording) by asking it to ‘go away’ via a chat message, providing

them a mechanism of consent. When a ZoomSensor detects it has entered a breakout room has been given co-host or recording permissions by the host, it automatically starts a local recording. When it leaves the room (either by the room session being ended by the host, or by being removed), it stops the recording and uploads the file to Firebase Storage. Ordinarily, the Zoom SDK would transcode the local raw recording into an mp4 once a recording session stops. To avoid significant load being placed upon the infrastructure, we intercept this process and instead upload the raw file. These are then processed in a sequential queue on the server, and the resulting mp4 is uploaded alongside original.

2.2 Scheduler

<https://gitlab.com/action-lab-aus/zoomsense/zoomsense-docker>

ZoomSense is designed using a ‘config driven’ philosophy, in which the expected configuration of ZoomSensors and their meetings is defined as a data object within the Firebase RTDB. A single instance of the containerised Scheduler NodeJS application runs on each VM within the ZoomSense cloud. It is responsible for adding ZoomSensors into meetings based on scheduling information within the Firebase RTDB, and for managing and reporting resources about each VM to aid in distributing ZoomSensors across the available resources. The manager initialises and then instructs each ZoomSensor (through instantiating a fresh instance of the container with dynamic ENV parameters) to join a specific Zoom meeting as an attendee. Multiple VMs can be provisioned on-demand and in parallel to respond to scheduling requirements. They can even be automatically decommissioned to save resources when no ZoomSensors are required, using existing infrastructure such as Kubernetes.

2.3 Server(less) Application

<https://gitlab.com/action-lab-aus/zoomsense/zoomsense-functions>

ZoomSense application logic is implemented as a set of ‘server-less’ Firebase Functions, negating the need to run a VM or host architecture. Meeting hosts are authenticated through the official Zoom OAuth SDK, allowing ZoomSense to obtain all relevant scheduling, meeting ID and joining requirements. When the host selects a meeting on the management interface, this information, plus the number of sensors required, are stored in the RTDB. When a meeting is scheduled to begin, the information is passed through the

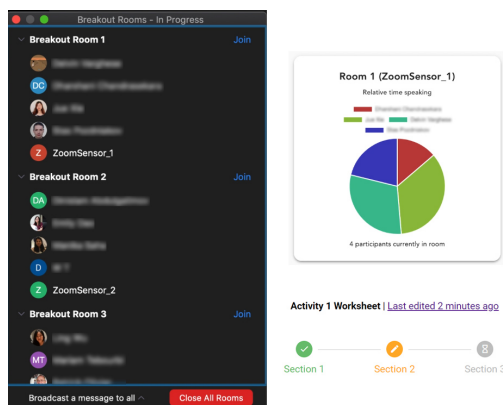


Figure 2: Left: ZoomSensors join breakout rooms alongside attendees; Right: the web client shows real-time data about a group’s activity, displaying members’ relative speaking time.

Firebase RTDB to the Docker Scheduler application of a VM that has the requisite capacity for the number of ZoomSensors requested. All configuration data related to a meeting—and any data related to interactions or participation within that meeting—is stored in the RTDB, where user access is limited to the host of the meeting, as authenticated by Zoom. This is controlled using native Firebase security policies.

2.4 Management Web Client

<https://gitlab.com/action-lab-aus/zoomsense/zoomsense-web-client>

The ZoomSense management web client is implemented as a static VueJS browser application. As ZoomSense is an infrastructure designed to support further augmentation, analysis and insight into the data that can be obtained from Zoom, the web client's primary purpose is to: i) support hosts in scheduling ZoomSensors to meetings; ii) obtain recordings of breakout sessions and iv) demonstrate and review captured data for both in real-time and for historical sessions. Users can schedule and monitor meetings augmented with ZoomSense by logging in using their existing Zoom credentials. Data security is maintained by only providing access to the meeting host, or those with whom the host has shared a unique anonymous sign-in link that can be generated within the UI—supporting monitoring and facilitation by multiple hosts or interaction with third party augmentations.

2.5 Augmentation

Deploying the full ZoomSense infrastructure requires access and resources from multiple external services: Zoom API (Developer Account), Firebase (min. Blaze Plan), Windows VM (min. 1) running Docker. However, the 'configuration driven' design of ZoomSense based around Firebase RTDB allows for easy augmentation of existing ZoomSense deployments in three patterns:

- (1) **System Wide Augmentation:** Gaining developer access to the Firebase RTDB and Functions of an existing deployment where new functionality for any meeting can be implemented.
- (2) **Host Meeting Augmentation:** Augmenting an individual host's meetings using just their own access credentials, as a meeting host has full access to the Firebase data corresponding to their meetings.
- (3) **Data Consumption Augmentation:** Using an anonymous access token created by the host in the management panel, any third party can be given read-only access directly to the RTDB to obtain data for a specific meeting.

We offer developer access to our live test infrastructure² to enable rapid development of new augmentations for research scenarios.

3 EXAMPLE AUGMENTATION SCENARIOS

The ZoomSense infrastructure provides a suitable foundation for rapid and flexible development of augmentations to Zoom. To showcase the diverse contexts in which ZoomSense can be (and is) used to develop prototypes with Zoom, we present a diverse selection of examples from our research collaborators.

²<https://zoomsense.io>

3.1 System Wide Augmentations

3.1.1 Multi-Modal Analytics.

Often, meetings on Zoom are conducted using additional external web-based services, such as shared documents to facilitate note-taking, group activities and agenda setting. In a teaching context, students in BO rooms may use these documents to record their progress through various activities. By automatically producing Google documents that map to known activities, it is possible to track the progress of each group through such shared documents, attributing specific edits to individual users, allowing for real-time insight that can inform tutors or facilitators. A single structured document template can be produced once by the tutor before the session and then be copied and re-distributed automatically by each ZoomSensor. Document edits can then be tracked in real-time so that an aggregate visualisation of progress can be presented to the tutor.

Research implementation using Firebase Functions and VueJS that creates structured Google Docs and visualises progress is available as part of the manager web client <https://gitlab.com/action-lab-aus/zoomsense/zoomsense-web-client>.

3.1.2 Supporting Attendee Agency & Automating Room Configuration.

As ZoomSensors can be assigned to the host role, they are able to manipulate breakout room configuration in real-time. This can support the automation of a meeting's room topology, allowing for complex configurations which would ordinarily be impractical due to the pressures placed upon the single host user. Examples could include the dynamic creation and attendance of breakout rooms according to current need or given variables (e.g. attendees' spoken languages), or the automation of participants moving between rooms: enabling complex scenarios such as 'round robin', where participants are paired off and swap partners every 5 minutes. By using a ZoomSensor as a proxy to the host controls available in the Zoom Client SDK, elevating it with host privileges can allow them to act *on behalf* of the host. As ZoomSensors can read and execute room manipulation commands from the Firebase RTDB, this can enable the control and configuration of breakout rooms in real-time via the RTDB.

Similarly, through the ability to proxy host-only meeting controls to Firebase RTDB, we can grant meeting attendees greater agency: attendees can communicate with a ZoomSensor to initiate features previously only accessible to the host. This can also be used in coordination with other ZoomSense features, such as data collection and audio recording. Once ZoomSensors are able to act with host privileges, previously restricted controls such as mute and breakout room control can be controlled via external logic, enabling attendees to request a move to a different room by sending commands to ZoomSense through dialog with a VP. This dialog can be processed relative to the current meeting structure and participation scaffolding, and translated into commands to be actioned by the VP with host privileges.

Reference implementation using Zoom Android SDK as an alternative ZoomSensor to manipulate BO rooms via Firebase <https://gitlab.com/action-lab-aus/zoomsense/zoomsense-bo-facilitator>.

3.2 Host Meeting Augmentations

3.2.1 Inter-Participant or Participant-Agent Dialogue.

While Zoom meeting hosts and attendees are currently only able to send and receive messages with those currently within their occupied breakout room, ZoomSensors can be used to expand the reach of these messages. As ZoomSensors are capable of sending and receiving arbitrary messages, they can act as a proxy interface to enable text-based communication between parties, regardless of their locations within the meeting. As message processing occurs within the cloud infrastructure, ZoomSensors can even enable the sending and receiving of messages with external platforms and technologies, such as Twitter, Reddit or SMS, brokering messages between these platforms and Zoom in real-time.

Beyond the system acting as a proxy to enable dialogue between multiple parties, ZoomSensors can also act as active conversation participants through the inclusion of natural dialogue processing frameworks on the server-side infrastructure. ZoomSensors can then act as ‘conversational agents’ through which access to additional services can be provided.

Student project implementation of a VP that facilitates ice-breaker activities in BO rooms <https://gitlab.com/action-lab-aus/zoomsense/zoomsense-icebreaker>.

3.2.2 Structured Meetings & Automatic Summaries.

In many contexts, the need arises to review and reflect on activity that occurred during a meeting at a later stage. Thus there is a need to record and store records of the content of a meeting and associated metadata for later analysis, presentation, and dissemination. Such metadata might include when participants move between rooms, interact with a ZoomSensor or engage with external tools, or when external factors influence activity in the meeting. Use of this data might include the automatic processing of recordings (e.g. transcription); redaction of recordings and transcripts according to user consent; interweaving of Zoom data and data collected from external platforms (e.g. Google Docs edits included in speech transcripts); automatic splicing and publication of recordings to external platforms for public consumption (e.g. paper presentations during virtual conferences). As recordings of breakout rooms are already available through ZoomSense, further processing of the capture metadata can support transcription, redaction according to consent, or integration with other data sources.

Research implementation using Firebase Functions where an external meeting agenda produces metadata that drives automatic video summaries of meetings <https://gitlab.com/action-lab-aus/zoomsense/zoomsense-functions/-/tree/feature/agenda>.

3.3 Data Consumption Augmentations

3.3.1 Live On-Screen Title Integration.

As real-time data can be obtained with minimal effort for specific meetings using the ‘data consumption’ augmentation pattern, a single ZoomSensor deployed into a live Zoom event can be used to support public dialog with the event management team. This data can then be displayed in real-time in an on-screen production tool to drive direct interaction between attendee content and on-screen graphics.

Production implementation using VueJS, where chat messages can be

displayed onto the host’s Zoom video as an overlay in real-time <https://github.com/tombartindale/opentitler>, <https://opentitler.web.app>.

3.3.2 Learning Activity Analytics.

Analytics systems can provide inferences about participants using data from the ZoomSensors, alerting the host when particular behaviours or actions are detected within breakout rooms. This analysis can take into account participants’ activities on connected third-party platforms and reduce the demands placed upon teachers during workshops featuring high student:educator ratios. When monitoring teaching sessions, tutors can view participants’ proportion of time speaking (using a pie chart representing active speaker data [Figure 2]), turn taking behaviour (using a network diagram) and each group’s progress through worksheets (using colour-coded icons). This data is pulled from the RTDB in real-time, and the UI provides a historical record of the data captured within each breakout session and main meeting. Tutors can either view real-time data, or ‘traverse’ back in time to observe patterns of behaviour or progress. While this data could be useful for gaining high-level insights into participation (such as identifying ‘bad behaviour’ like silence within a group), more specific visualisations could be developed that are linked contextually to specific pedagogical scenarios, such as identifying and nudging participants who are not contributing towards (or are dominating) group discussions, or identifying and nudging groups who have discussed a given issue, but haven’t documenting their progress on shared worksheets.

Research implementation using Firebase Functions and VueJS, visualising live and historical activity data specifically designed to support tutors managing online classes <https://gitlab.com/Stalkcomrade/zoomsense-web-client>.

4 CONCLUSION

We introduced ZoomSense: an open-source infrastructure to support the rapid development of augmentations to Zoom. We discussed the potential for using ZoomSense with example implementations, and how ZoomSense can enable research into new forms of participant interaction for online video communication. ZoomSense is an active and collaboration project. Researchers at University College London, New York University, Carnegie Mellon University and Monash University are all currently maintaining active installations of the ZoomSense infrastructure, and are developing a variety of augmentations. We plan to continue development of the ZoomSense platform, and invite others to contribute towards the open-source project and join us in exploring new models for remote participation with Virtual Participants.

REFERENCES

- [1] Stephanie Tonneson. 2020. What Explains Zoom Video’s Success During the Coronavirus? <https://blog.zoominfo.com/zoom-video-growth-coronavirus/>. (Accessed on 04/08/2020).
- [2] Eric S. Yuan. 2020. A Message to Our Users - Zoom Blog. <https://blog.zoom.us/a-message-to-our-users/>. (Accessed on 05/08/2020).
- [3] Qi Zhou, Wannapon Suraworachet, Stanislav Pozdniakov, Roberto Martinez-Maldonado, Tom Bartindale, Peter Chen, Dan Richardson, and Mutlu Cukurova. 2021. Investigating Students’ Experiences with Collaboration Analytics for Remote Group Meetings. In *Artificial Intelligence in Education*, Ido Roll, Danielle McNamara, Sergey Sosnovsky, Rose Luckin, and Vania Dimitrova (Eds.). Springer International Publishing, Cham, 472–485.
- [4] Zoom. 2019. Zoom for Higher Education. <https://zoom.us/docs/doc/Zoom%20for%20Higher%20Education.pdf>. (Accessed on 05/08/2020).